Module 3 — Watch the Research Agent (35 min, instructor demo)

> **You watch; the instructor drives.** Claude (an AI assistant) runs on the presenter's laptop. It reads a research doc and your hunts, opens a new hunt from the research, then **writes** the Splunk searches one at a time. In this module Claude is **not connected to Splunk**: the presenter reads each search, pastes it into Splunk Web, and pastes the result rows back to Claude. (Next module, Claude runs them itself.) You don't need an LLM key. If you want, paste the same searches into your own Splunk (time picker: **Last 30 days**).
> **Level coverage: L3.** Starting point: **H-0005**, the beacon you left open in Module 1. Ends with **H-0007** on the presenter's screen.

Opening — research is the step before the hunt (3 min)

Say: Modules 1 and 2 were hunting: you had a hypothesis and tested it. Research is the step before that. It reads what's known about a behavior and tells you what to look for. In ATHF they're two objects: a research doc (`R-0001`) proposes, a hunt (`H-0007`) tests, and the hunt links back to the research that started it.

Say: Today Claude writes the searches and I run them. That's L3: the agent drafts, a human reviews and executes. In Module 4 Claude runs them itself.

Beat 1 — Research the open thread (5 min)

Say: H-0005 is still open. We know WS-BUILD-01 beacons every ~5 minutes. We don't know what keeps it running, or what else the actor runs. The events are in Splunk; what we're missing is *what to look for*.

Show (on screen; don't run it, it was run before class):

```
athf research new \
  --topic "Periodic outbound to a CDN-lookalike domain after a build-CLI "\
"install on a developer workstation: how the node beacon persists "\
"through developer tools like git, npm or the IDE, and how to find "\
"related attacker infrastructure" \
  --technique T1071.001 \
  --depth basic
```

Say: This takes about 30 seconds and four LLM calls, so I ran it before class. Five steps: how does this normally work, how do attackers abuse it, which log fields show it, have we hunted it before, then a summary. The topic reuses H-0005's words on purpose: step four searches your hunts, and it only finds H-0005 if the words match.

Following along? Read `research/R-0001.md` in your repo. To run research yourself later, `athf research new` needs an Anthropic or OpenAI API key, AWS Bedrock credentials, or a local Ollama. A Claude subscription alone won't work.

Show: open `research/R-0001.md` in the editor. Read four lines:

1. **Related Work:** `H-0005` . It read *your* hunt.
2. **System Research:** *"...core.hooksPath pointing to a shared directory..."* → persistence.

3. **Tradecraft:** *"...unexpected outbound Node.js process network connections from IDE or git-related parent processes"* → the search.
4. **Tradecraft:** *"...same ASN/registrar ... share JA3/JA3S fingerprints"* → infrastructure reuse. If we don't have JA3: resolve the server name to its IP and look next door.

Guiding question (interpretation): *Which claims can you check in our data, and which are guesses? Look at "Data Source Availability" and the beacon interval it expects.*

Why the topic names git: a dev box runs git, npm and an IDE all day. Those are the tools a backdoor would hide in.

Beat 2 — Open H-0007 from the research (5 min)

Say: We create the hunt first, before any search. The research gives us the hypothesis and what to look for; the hunt is where the searches and results will land. And it gives "update the hunt" one obvious meaning later: H-0007, not H-0005.

Type (to Claude):

```
Read research/R-0001.md and hunt H-0005 (the open beacon thread). Before we
search anything, create the hunt this research starts, H-0007. Use the athf
hunt new CLI in the terminal (not the MCP tool, so the research link is
written both ways), with --non-interactive and --research R-0001. Title:
"Git-hook persistence relaunches the beacon; lookalike C2 domain on the same
hosting". Technique T1546; tactics persistence and command-and-control;
platform windows; data sources edr, proxy and network. Fill --hypothesis,
--threat-context, --actor, --behavior, --location and --evidence from the
research and H-0005, in plain text (no backticks or $ inside the quotes).
Leave --hunter off (the config has it). Then, in the new file, fill OBSERVE
(what normal looks like, what suspicious looks like, expected observables)
and set status to in-progress. Leave CHECK and KEEP as the template. Run athf
hunt validate. Only create the hunt — don't write any searches yet.
```

Show:

1. The `athf hunt new` command Claude builds: `--non-interactive --research R-0001`.
2. Its output: `Created H-0007` and `Linked H-0007 to research R-0001`.
3. The new file: `spawned_from: R-0001`, the hypothesis and ABLE table in LEARN, OBSERVE filled, CHECK and KEEP still the template.
4. `research/R-0001.md` now lists `linked_hunts: ['H-0007']`.

Beat 3 — One search at a time: what starts the beacon? (9 min)

Say: Now the loop. I ask Claude for **one** search. I read it, paste it into Splunk, run it, and paste the result rows back. Claude says what they mean, then writes the next one. One search per turn keeps every step small enough for me, and you, to check.

The loop (every turn after the first):

```
Result from Splunk:

<paste the result rows, header included>

What does this tell us, in two or three lines? Then write the next single
search: <the next search, from this handout>. Same rules: don't run it,
one code block, one line of why. Only this one search — don't
investigate further.
```

S1 — What did git launch, on every host?

Type (to Claude):

```
Now the first search, for the research's first lead: git hooks, and node
started from git-related parent processes. You can't reach Splunk in this
module: write the search, don't run it. I'll paste it into Splunk and paste
the result back to you. Write ONE search: what git launched, on every host
(eCAR process creates whose parent image is git.exe), as a table with _time,
hostname, principal, pid, ppid, image and command line. Reuse the index,
sourcetype and field names from our hunts' CHECK sections. No subsearches, no
comments in the SPL, one code block, one line of why. Only write this one
search — don't investigate further.
```

Claude writes (review it, then paste into Splunk):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\git.exe"
| table _time hostname principal pid ppid properties.image_path properties.command_line
```

S2 — What else ran from a hooks folder on that host?

Type (to Claude): `Result from Splunk:` , the S1 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: everything on that host whose command line mentions hooks, in time
order, with pid, ppid, parent image and command line. Same rules: don't run
it, one code block, one line of why. Only this one search — don't
investigate further.
```

Claude writes:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*hooks*"
| sort _time
| table _time pid ppid properties.parent_image_path properties.command_line
```

S3 — Is that node process the beacon from H-0005?

Type (to Claude): `Result from Splunk:` , the S2 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: node's outbound flows on that host (eCAR FLOW, OUTBOUND; filter on
the node image, not a pid), counted by pid and destination IP and port, with
the first time. Same rules: don't run it, one code block, one line of why.
Only this one search — don't investigate further.
```

Claude writes:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW properties.direction=OUTBOUND
hostname="WS-BUILD-01" properties.image_path="*node*"
| stats count min(_time) as first by pid properties.dst_ip properties.dst_port
```

What it means. Compare with Module 1's front door.

Guiding question (decision): *M1 found node launching a shell after an install. What's the same idea here, and what changed?*

Beat 4 — Infrastructure reuse: does the actor run more than one domain? (9 min)

Say: Research's second lead: lookalike domains share hosting and TLS fingerprints (JA3). Before we pivot on a field, check we have it. Claude can't look, so it writes the check.

S4 — Do we have JA3?

Type (to Claude): `Result from Splunk:` , the S3 row, then:

```
What does this tell us, in two or three lines? That closes the first lead.
The research's second lead: lookalike domains share hosting and JA3/JA3S TLS
fingerprints. Write the next single search: list the fields our Zeek SSL
data (sourcetype bro:ssl:json) actually has, so we can see whether ja3
exists. Same rules: don't run it, one code block, one line of why. Only this
one search — don't investigate further.
```

Claude writes:

```
index=thrunt sourcetype="bro:ssl:json" | fieldsummary | table field
```

S5 — Where is the C2 hosted?

Type (to Claude): `Result from Splunk:` , the S4 field list, then:

```
What does this tell us, in two or three lines? Then write the next single
search: the server IP that cdn-packmule-telemetry.net resolves to in Zeek
SSL, with a count. Same rules: don't run it, one code block, one line of
why. Only this one search — don't investigate further.
```

Claude writes:

```
index=thrunt sourcetype="bro:ssl:json" server_name="cdn-packmule-telemetry.net"
| stats count by id.resp_h
```

S6 — What else is next door?

Type (to Claude): `Result from Splunk:` , the S5 row, then:

```
What does this tell us, in two or three lines? Then write the next single
search: every server_name in Zeek SSL on the addresses right next to that
IP (a /30, using cidrmatch; inside where, single-quote the dotted field as
'id.resp_h', but leave it unquoted in stats), with counts and IPs. Same
rules: don't run it, one code block, one line of why. Only this one search —
don't investigate further.
```

Claude writes (with the IP from S5):

```
index=thrunt sourcetype="bro:ssl:json"
| where cidrmatch("<C2 IP>/30", 'id.resp_h')
| stats count by server_name id.resp_h
```

S7 — Who talks to it?

Say: Zeek only sees the proxy, so ask the proxy log.

Type (to Claude): `Result from Splunk:` , the S6 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: in the proxy log, which internal client and user connected to the
new domain, with counts and first and last time. Same rules: don't run it,
one code block, one line of why. Only this one search — don't investigate
further.
```

Claude writes (with the new domain from S6):

```
index=thrunt sourcetype="apache:access:json" server="<the new domain>"
| stats count min(_time) as first max(_time) as last by client user
```

What it means. Two lookalikes, next-door IPs, one client.

Guiding question (interpretation): *Why is "same hosting block" stronger evidence of the same actor than a similar name? When would it be weak?*

Beat 5 — Claude updates H-0007 (3 min)

Say: In Module 1 you wrote the hunt by hand. Here the assistant fills the hunt it opened, from the searches it wrote and the rows I pasted back.

Type (to Claude): `Result from Splunk:` , the S7 rows, then:

```
What does this tell us, in two or three lines? Then update H-0007 with what
we did: in CHECK, the seven searches I ran (one code block each under Hunting
Queries) and one Query Notes bullet per search with what it returned; in
KEEP, an Executive Summary and one suspected Findings row per finding (the
git-hook chain on WS-BUILD-01, and the second domain), plus Follow-up
Actions. Set status to completed and run athf hunt validate. Only edit
H-0007 — no new searches.
```

Show:

1. CHECK: the seven searches, each with what it returned.
2. KEEP: two `suspected` Findings rows.
3. `athf hunt validate` passes.

Guiding question (reflection): *Why `suspected`, not `confirmed`, even though every search lined up?*

Recap (1 min)

Guiding question (reflection): *Module 1 took 30 minutes by hand: seven searches, three copied IDs, one process tree and one open beacon. What did the last half hour produce, and what did the agent actually contribute?*

Stop here — on the presenter's screen

```
athf hunt list
```

```
athf research list
```

Next: Module 4. Claude runs the searches itself, and uses your hunt history to decide what to hunt next.